DISTRIBUTED MOVIE STREAMING DATABASE

System Architecture, Replication & Sharding Design

A Distributed Database Approach to Movie & TV Show Metadata Management

University of Karachi
UBIT — Department of Computer Science

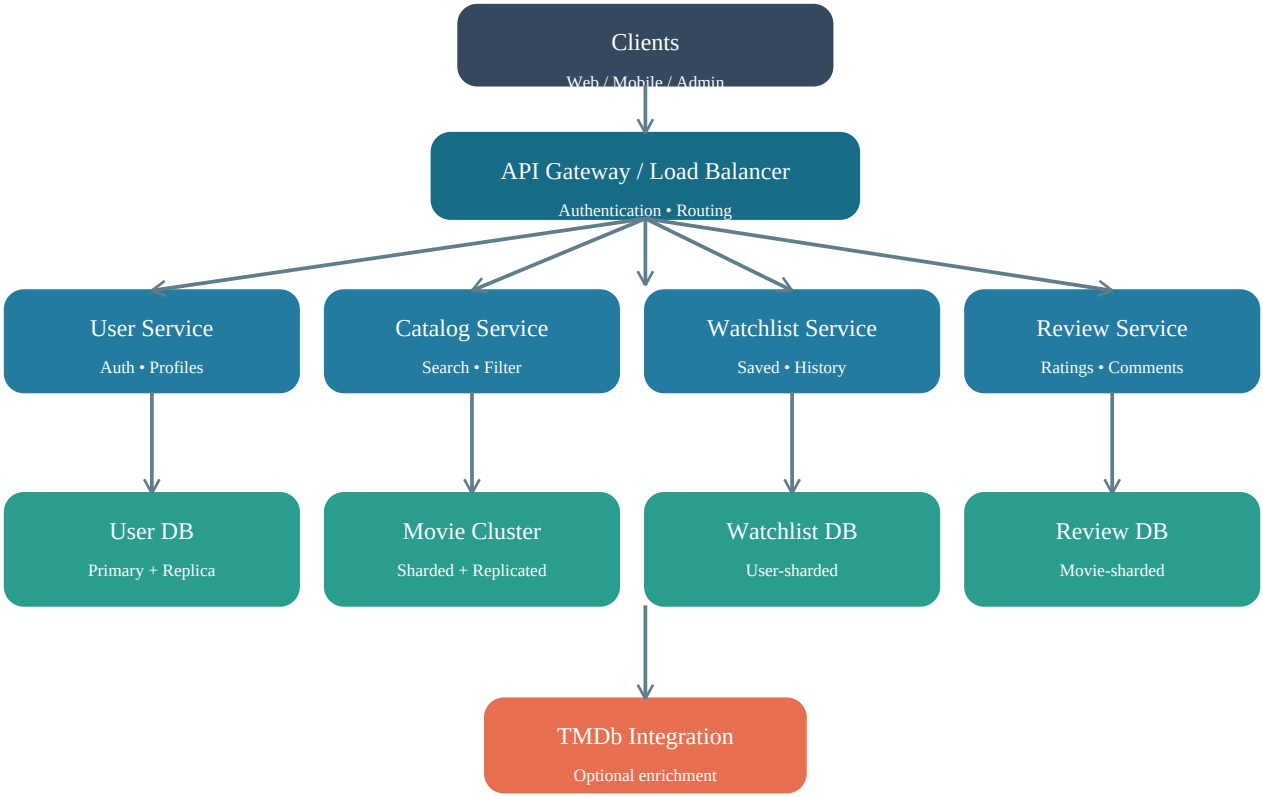
Course: Distributed Database Systems (CS 606)
Submitted to: Sir Umair Ahmed

Prepared as an enhanced architecture supplement to the project proposal.

1. Architecture Overview

The proposed platform is a Netflix-like metadata system. It manages movie and TV-show information, search, user accounts, watchlists, reviews, ratings, and administrative operations. Actual video delivery is outside the project scope. The architecture below preserves the proposal's four independent databases while adding a service layer, routing, replication, sharding, caching, and asynchronous synchronization.

High-Level System Architecture



Request flow: Client → API Gateway → relevant service → database cluster → response. The gateway can distribute traffic across multiple service instances, while each service owns its data domain.

2. Core Components

Layer	Responsibility
Client Layer	Web/mobile users and administrators submit searches, manage watchlists, post reviews, and maintain profiles.
API Gateway / Load Balancer	Single entry point; validates requests, applies authentication, rate limits traffic, and routes requests to services.
Service Layer	User, Catalog, Watchlist, Review, and Admin services implement business logic and isolate module responsibilities.
Data Layer	Independent databases allow each workload to scale and evolve separately.
Cache / Search Optimization	Optional Redis cache and search indexes reduce repeated catalog reads and improve search latency.

Integration Layer	Optional TMDb connector enriches posters, synopses, cast, and other metadata.

3. Data Ownership and Cross-Database References

Each service owns its database. Cross-database relationships are represented through stable IDs rather than database-level foreign keys. For example, Review records store **userId** and **movieId**; Watchlist records store **userId** and **contentId**. This avoids tight coupling between independently deployed databases.

Database	Primary data	Scaling approach
User DB	Users, credentials, profiles, preferences	Primary–replica; shard later by userId if needed
Movie DB	Titles, genres, cast, directors, languages, popularity	Catalog shards + replicas; read-heavy
Watchlist DB	Favorites, saved content, viewing history	Hash sharding by userId
Review DB	Ratings, comments, moderation status	Hash/range sharding by movieId

4. Sharding Design

Sharding horizontally partitions a large dataset across multiple database nodes. Unlike replication, which copies the same data to multiple nodes, sharding distributes different subsets of data across nodes. The design below adds sharding where data volume and request distribution are expected to grow.

4.1 Movie Catalog Sharding

Recommended approach: **hash-based sharding using contentId**. A routing layer calculates the shard from the content ID, for example: **shard = hash(contentId) mod N**. This provides relatively even distribution and avoids placing all popular titles in one shard. Each shard has one primary and one or more replicas.

Alternative: genre- or language-based partitioning may simplify some filters, but it can create uneven data distribution and cross-shard queries. Therefore, it is better treated as a logical index or secondary search structure rather than the primary shard key.

4.2 Watchlist Sharding

Shard by **userId**. All watchlist entries for one user remain on the same shard, making add, remove, and fetch operations efficient. A request router uses a consistent-hash ring so new shards can be added with limited data movement.

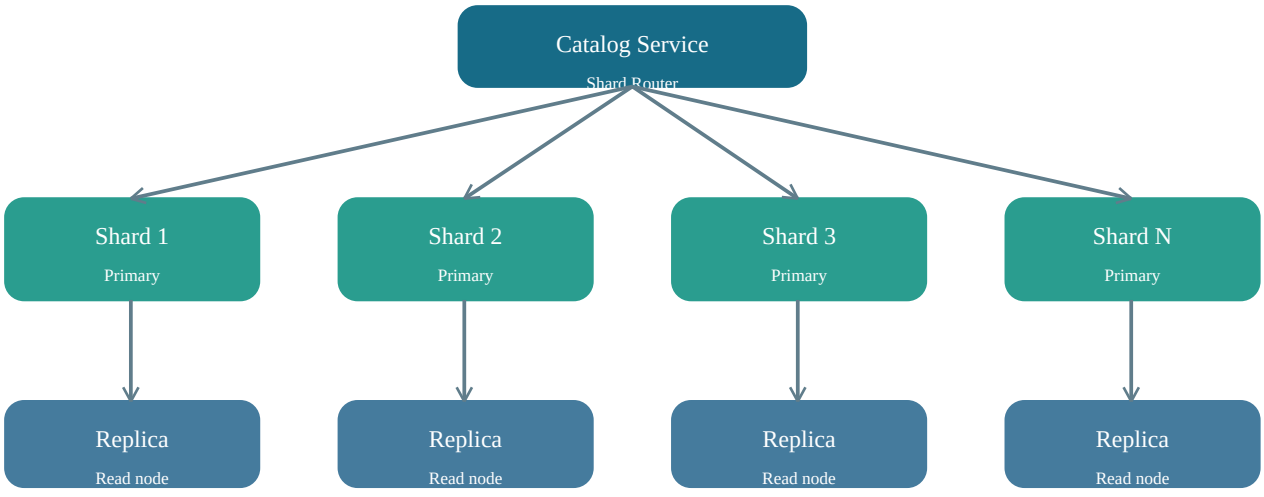
4.3 Review Sharding

Shard by **movieId** because reviews are commonly fetched per movie. This keeps a movie's review workload localized. To avoid hot shards for highly popular titles, the system can add a secondary bucket, such as **hash(movieId + bucket)**, if traffic becomes extreme.

4.4 Shard Router

The shard router is part of the service/data-access layer. It maintains a shard map, routes reads and writes to the correct shard, and hides physical node locations from application code. A metadata/configuration store records shard ownership and health.

Sharding Layout



Read path: Router → shard replica when permitted. **Write path:** Router → shard primary → replication log/event → replicas.

5. Replication and Consistency

The original proposal identifies the Movie Database as the most read-intensive component and recommends replication across multiple nodes. This enhanced design keeps that approach and applies replication within each shard.

Replication policy

• **Primary–replica replication:** each shard has one write primary and one or more read replicas.

• **Read scaling:** search, filtering, sorting, and catalog browsing are routed to healthy replicas.

• **Write handling:** admin catalog updates are sent to the primary and propagated to replicas.

• **Consistency:** asynchronous replication provides lower write latency but may briefly return stale data. Stronger consistency can be requested for critical admin verification.

• **Failover:** if a primary fails, a healthy replica is promoted and the shard map is updated.

• **Review and watchlist writes:** writes go to the owning shard primary to preserve ordering for a user or content item.

6. Search, Cache, and Query Strategy

Catalog search can be optimized using indexes on title, genre, actor, director, release year, language, rating, popularity, and release date. A cache stores frequently requested title details, category pages, and popular search results. Cache entries should be invalidated or refreshed after catalog updates.

For searches that cannot be routed to one shard, the Catalog Service performs a controlled scatter–gather query: it queries relevant shards in parallel, merges results, applies global sorting, and returns a paginated response. This should be limited because broad cross-shard searches are more expensive.

7. Availability and Failure Handling

• **Service failure:** the load balancer routes traffic to another healthy service instance.

• **Database-node failure:** replicas continue serving reads; primary failover restores writes.

• **Shard failure:** only the affected shard’s data is degraded, limiting the blast radius.

• **Review DB failure:** movie browsing can continue while review functionality is temporarily unavailable.

• **TMDb failure:** the platform continues using locally stored metadata.

• **Retries:** use bounded retries with exponential backoff for transient failures.

• **Backups:** take periodic snapshots and retain replication logs for point-in-time recovery.

8. Recommended Technology Mapping

Component	Suggested implementation	Reason
API / Services	Node.js + Express or NestJS	Simple modular service development
User DB	PostgreSQL or MongoDB	Reliable identity/profile storage
Movie DB	MongoDB sharded cluster	Flexible metadata and horizontal scaling
Watchlist DB	MongoDB / PostgreSQL	Efficient user-keyed access
Review DB	MongoDB sharded cluster	High-volume content-keyed records
Cache	Redis	Low-latency popular-content caching
Search (optional)	Elasticsearch / OpenSearch	Advanced full-text search and ranking
Messaging (optional)	Kafka / RabbitMQ	Async replication events and analytics

9. End-to-End Example Flows

Movie search: Client → API Gateway → Catalog Service → cache lookup → shard router → relevant movie-shard replicas → merge/sort if required → response.

Add to watchlist: Client → API Gateway → Watchlist Service → route by userId → watchlist-shard primary → replicate → success response.

Submit review: Client → API Gateway → Review Service → validate user and content IDs → route by movieId → review-shard primary → replicate → update aggregate asynchronously.

Admin catalog update: Admin → API Gateway → Admin/Catalog Service → movie-shard primary → replication to replicas → cache invalidation → search-index update.

10. Final Architecture Summary

The final design combines **service-level data ownership**, **horizontal sharding**, and **replication**. Movie metadata is distributed across catalog shards and replicated for high read throughput. Watchlists are sharded by userId, reviews by movieId, and the User Database remains independently managed. The architecture supports independent scaling, fault isolation, lower read latency, controlled cross-shard querying, and future growth without changing the core application model.

Key distinction: Partitioning separates data by domain (User, Movie, Watchlist, Review). Sharding splits a large domain horizontally across nodes. Replication copies shard data to additional nodes for availability and read scaling.

11. Assumptions and Scope

This document expands the supplied proposal with an implementable architecture and sharding strategy. The proposal itself specifies four independent databases and Movie Database replication; the detailed shard keys, shard router, cache, failover behavior, and technology mapping are architectural additions intended to make the design complete and presentation-ready. Actual video streaming, CDN delivery, and media-file storage remain out of scope.